

Session 7 — Part 0: Asynchronous Communication with Kafka

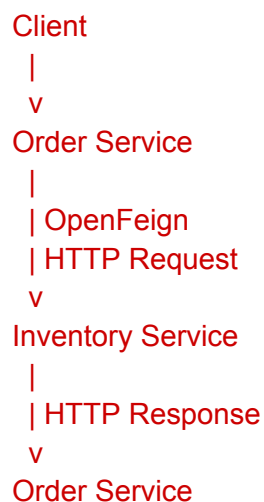
From OpenFeign to Event-Driven Communication

Goal: Before implementing the Saga Pattern, we first learn how two microservices communicate asynchronously using Apache Kafka.

1. Where We Left Off in Session 6

In Session 6, we implemented **synchronous** inter-service communication.

Our `order-service` communicates with `inventory-service` using OpenFeign:



The important characteristic is:

Order Service waits for Inventory Service to answer before continuing.

This is **Synchronous Communication**.

Request



Wait



Response



Continue

The caller **cannot continue until** the downstream service responds or the call fails

For an inventory check, this can make sense:

Order Service:

"Is this product available?"

Inventory Service:

"Yes" or "No"

The answer is required immediately to make the next business decision.

2. A New Requirement: Send an Order Confirmation

Now we have a new requirement:

After an order is created, send a **confirmation notification** to the customer.

One possible design is another **synchronous** call:

Order Service



| OpenFeign



Notification Service



| Send Email



Order Service waits...

But ask an important question:

Does Order Service need to wait for the email to be sent before completing the request?

Usually, the answer is:

No.

The order may already be successfully created even if:

- the email provider is temporarily unavailable;
- sending the email takes several seconds;
- the Notification Service is temporarily restarting.

The notification is a **reaction to an event that already happened**.

Instead of directly calling Notification Service, Order Service can publish:

OrderCreatedEvent

Notification Service can react **asynchronously**.

Order Service

|

| OrderCreatedEvent

v

Kafka

|

v

Notification Service

|

v

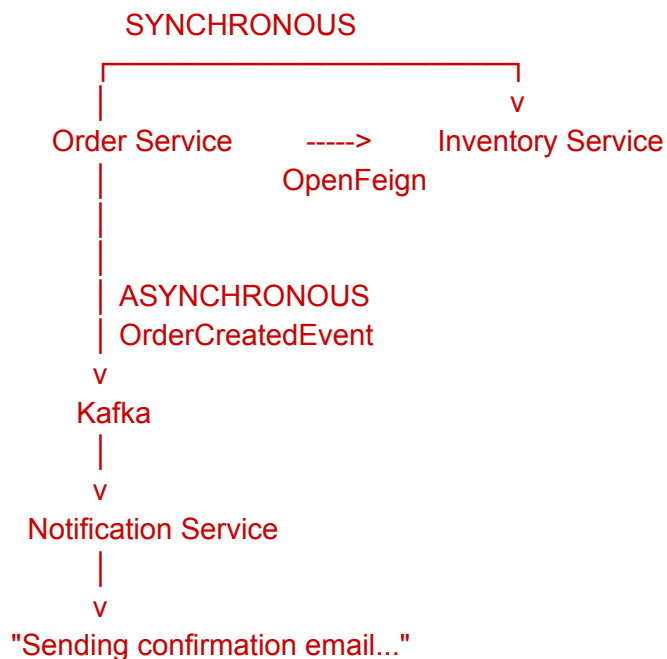
Send Email

Order Service **does not** need to **wait** for Notification Service.

This is our first **asynchronous** communication flow.

3. What We Will Build

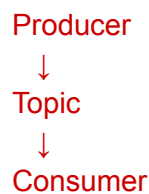
By the end of this section, our platform will contain:



This example is intentionally simple.

This is not a Saga yet.

We are first learning the mechanics of **asynchronous** communication:



After that, we will use the same Kafka concepts to implement a **distributed business transaction** using the **Saga** Pattern.

4. Command Thinking vs Event Thinking

This is an important mental shift.

With synchronous REST communication, we often think in commands:

Order Service

|

| "Check inventory"

v

Inventory Service

or:

Order Service

|

| "Send notification"

v

Notification Service

The caller **knows exactly which service it is calling**.

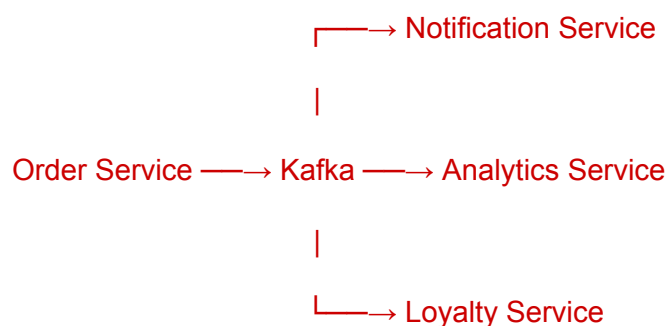
With **event-driven** communication, a service can **publish a fact**:

OrderCreated

The event means:

Something already happened in the domain.

The producer does **not** necessarily need to know every service interested in that event.



Order Service **publishes once**:

OrderCreatedEvent

Multiple consumers may react independently.

5. The First Async Example — Before Saga

Before using Kafka for a distributed transaction, we first need to **understand Kafka** as a communication mechanism.

We will begin with the simplest possible platform example:

Order Service

|

| publishes

v

OrderCreatedEvent

|

v

Kafka Topic: order-created

|

| consumed by

v

Notification Service

|

v

Send confirmation notification

This is:

Asynchronous Communication

It is **not** yet a Saga.

It does **not** coordinate a distributed transaction.

It simply **communicates that something happened**.

6. The Event

A simple **event** might look like:

```
public record OrderCreatedEvent(  
    Long orderId,  
    Long customerId,  
    BigDecimal totalAmount  
) {  
}
```

Notice the naming:

OrderCreated

not:

CreateOrder

Why?

Because an **event** represents:

A fact that already happened.

Examples:

- OrderCreated
- PaymentCompleted
- InventoryReserved
- OrderCancelled

These are **past-tense** domain facts.

7. Kafka Mental Model

For today's session, begin with only three concepts:

Producer

|

| sends event

v

Topic

|

| stores / distributes event

v

Consumer

Using our platform:

Producer: Order Service

Topic: order-created

Consumer: Notification Service

The flow is:

Order Service

|

| KafkaTemplate.send(...)

v

order-created topic

|

| @KafkaListener

v

Notification Service

8. Producer — Publishing an Event

In Spring Boot, a service can **publish** an event using `KafkaTemplate`.

Conceptually:

`@Service`

```
public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventPublisher(
        KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publish(OrderCreatedEvent event) {

        kafkaTemplate.send(
            "order-created",
            event.orderId().toString(),
            event
        );
    }
}
```

The important line is:

`kafkaTemplate.send(...)`

Order Service **publishes** an event to a topic.

It does **not** call:

```
notificationService.sendEmail(...)
```

It does **not** use:

```
@FeignClient(name = "notification-service")
```

It **publishes** a **domain fact**:

```
OrderCreated
```

9. Consumer — Reacting to an Event

Notification Service can **listen** for that event:

```
@Component
```

```
public class OrderCreatedListener {  
  
    @KafkaListener(  
        topics = "order-created",  
        groupId = "notification-service"  
    )  
  
    public void handle(OrderCreatedEvent event) {  
  
        System.out.println(  
            "Sending notification for order: " + event.orderId()  
        );  
    }  
}
```

The consumer reacts when the event arrives.

The producer does not wait for this method to finish.

10. Start Kafka with Docker

Kafka will run as infrastructure for our microservices platform.

Add Kafka to the existing `docker-compose.yml`.

For a modern local development environment, Kafka can run in **KRaft mode**, which does not require ZooKeeper.

Example:

services:

```
kafka:
  image: apache/kafka:latest
  container_name: kafka
  ports:
    - "9092:9092"
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
      CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
```

Course Note: The exact Docker image version should be pinned in the course repository rather than using `latest`. The repository version is the source of truth for the lab.

Start Kafka:

```
docker compose up -d kafka
```

Verify:

```
docker compose ps
```

Kafka should be running.

. Kafka Mental Model

For now, remember only three main concepts.

Producer

|

| publishes a message

v

Topic

|

| message is consumed

v

Consumer

In our platform:

Producer:

Order Service

Topic:

order-created

Consumer:

Notification Service

The complete flow is:

Order Service

|

| KafkaTemplate.send(...)

v

order-created

|

| @KafkaListener

v

Notification Service

11. Step 1 — Update Order Service

We begin with `order-service` because it already exists from Session 6.

The Order Service will become our first Kafka **Producer**.

Its responsibility is:

Create Order



Publish OrderCreatedEvent

11.1 Add Spring Kafka Dependency

Open:

[services/order-service/pom.xml](#)

Add:

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

Spring Boot manages the compatible Spring Kafka version.

Do not manually add a version unless the project explicitly requires one.

11.2 Create the OrderCreatedEvent

Create:

[event/OrderCreatedEvent.java](#)

Example:

```
package com.microservicespro.order.event;

import java.math.BigDecimal;

public record OrderCreatedEvent(
```

```
        Long orderId,  
        Long customerId,  
        BigDecimal totalAmount  
    ){  
    }
```

This event represents a fact:

An order was created.

Notice the name:

OrderCreated

not:

CreateOrder

A useful naming convention is:

Command:

CreateOrder

Event:

OrderCreated

A command asks for something to happen.

An event describes something that already happened.

11.3 Configure Kafka in Order Service

Open:

`application.yml`

Add the Kafka bootstrap server and producer serialization configuration:

`spring:`

`kafka:`

`bootstrap-servers: localhost:9092`

```
producer:  
key-serializer: org.apache.kafka.common.serialization.StringSerializer  
value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

The important configuration is:

bootstrap-servers

It tells the application where Kafka is running.

For local development:

localhost:9092

11.4 Create the Kafka Producer

Create:

[messaging/OrderEventPublisher.java](#)

```
package com.microservicespro.order.messaging;  
  
import com.microservicespro.order.event.OrderCreatedEvent;  
import org.springframework.kafka.core.KafkaTemplate;  
import org.springframework.stereotype.Component;  
  
@Component  
public class OrderEventPublisher {  
  
    public static final String ORDER_CREATED_TOPIC = "order-created";  
  
    private final KafkaTemplate<String, Object> kafkaTemplate;  
  
    public OrderEventPublisher(  
        KafkaTemplate<String, Object> kafkaTemplate) {  
        this.kafkaTemplate = kafkaTemplate;  
    }  
  
    public void publishOrderCreated(OrderCreatedEvent event) {  
  
        kafkaTemplate.send(  
            ORDER_CREATED_TOPIC,
```

```

        event.orderId().toString(),
        event
    );

    System.out.println(
        "Published OrderCreatedEvent for order: "
        + event.orderId()
    );
}
}

```

The important operation is:

```
kafkaTemplate.send(...)
```

Conceptually:

```

Order Service
  |
  | send
  v
Kafka Topic

```

The first argument is the **topic**:

```
order-created
```

The second is the **message key**:

```
orderId
```

The third is the **event payload**:

```
OrderCreatedEvent
```

11.5 Publish the Event After Creating the Order

Now connect the producer to the existing Order Service flow.

Conceptually:

Create Order



Save Order



Publish OrderCreatedEvent



Return Response

Inject `OrderEventPublisher` into the existing service responsible for creating orders.

Example:

```
@Service
public class OrderService {

    private final OrderEventPublisher eventPublisher;

    public OrderService(
        OrderEventPublisher eventPublisher) {
        this.eventPublisher = eventPublisher;
    }

    public Order createOrder(CreateOrderRequest request) {

        // Existing Session 6 business logic
        // 1. Validate request
        // 2. Check inventory using OpenFeign
        // 3. Create and save the order

        Order savedOrder = saveOrder(request);

        OrderCreatedEvent event =
            new OrderCreatedEvent(
                savedOrder.getId(),
                savedOrder.getCustomerId(),
                savedOrder.getTotalAmount()
            );

        eventPublisher.publishOrderCreated(event);

        return savedOrder;
    }
}
```

Adapt this code to the actual `Order` model and method names in the course repository.

The important sequence is:

Order saved successfully



OrderCreatedEvent published

11.6 What Happens Now?

At this point:

Order Service



| OrderCreatedEvent



Kafka

But nobody is listening yet.

Kafka can receive the event, but we need a consumer.

Now we create:

Notification Service

11.7 Step 2 — Create Notification Service

Go to Spring Initializr and create a new Spring Boot application with the same Java and Spring Boot baseline used by the course repository.

Project information:

Artifact:

`notification-service`

Name:

`notification-service`

Java:

`21`

Use the same package naming convention as the rest of the platform.

Add the dependencies required for this service:

[Spring for Apache Kafka](#)
[Spring Boot Actuator](#)

If the service must register with the platform's Eureka Server, also add:

[Eureka Discovery Client](#)

If it loads configuration from the Config Server, also add:

[Config Client](#)

The exact dependency set should remain consistent with the current platform architecture.

Extract the generated project under:

[services/notification-service](#)

11.8 Notification Service Maven Dependencies

The important dependency for this section is:

```
<dependency>  
  <groupId>org.springframework.kafka</groupId>  
  <artifactId>spring-kafka</artifactId>  
</dependency>
```

For Actuator:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

If Eureka is used:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

11.9 Create the Event Contract in Notification Service

For this first teaching example, create the same event structure in Notification Service:

```
package com.microservicespro.notification.event;

import java.math.BigDecimal;

public record OrderCreatedEvent(
    Long orderId,
    Long customerId,
    BigDecimal totalAmount
){
}
```

The producer and consumer must agree on the event contract.

Conceptually:

Producer Contract
=
Consumer Contract

Later in the course, contract evolution and compatibility require more careful design.

For now, both sides use the same JSON structure.

11.10 Configure the Kafka Consumer

Open the Notification Service:

application.yml

Example:

```
server:
  port: 8084

spring:
  application:
    name: notification-service

kafka:
  bootstrap-servers: localhost:9092

  consumer:
    group-id: notification-service
    auto-offset-reset: earliest
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer

  properties:
    spring.json.trusted.packages: ""
```

For this teaching example:

group-id = notification-service

identifies the consumer group.

The JSON deserializer converts the Kafka message back into a Java object.

`trusted.packages: "*" is convenient for a controlled training environment.`
In a production system, restrict trusted packages rather than accepting every package.

11.11 Create the Kafka Consumer

Create:

messaging/OrderCreatedEventListener.java

```
package com.microservicespro.notification.messaging;
```

```
import com.microservicespro.notification.event.OrderCreatedEvent;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
```

```
@Component
public class OrderCreatedEventListener {

    @KafkaListener(
        topics = "order-created",
        groupId = "notification-service"
    )
    public void handle(OrderCreatedEvent event) {

        System.out.println(
            """
            =====
            NOTIFICATION SERVICE
            Sending order confirmation email...
            Order ID: %s
            Customer ID: %s
            Total Amount: %s
            =====
            """).formatted(
                event.orderId(),
                event.customerId(),
                event.totalAmount()
            )
        );
    }
}
```

For now, we are not integrating a real email provider.

The log message represents:

Send Email

Our goal is to learn asynchronous service communication.

11.12 Run the Complete Flow

Start Kafka:

`docker compose up -d kafka`

Start the services required by the current platform.

Then start:

Order Service

Notification Service

Create a new order using the existing Order API.

Conceptually:

`POST /api/v1/orders`

The exact request body depends on the current Order API implementation.

The expected flow is:

1. Client creates an order
2. Order Service performs its existing business logic
3. Order is created
4. Order Service publishes OrderCreatedEvent
5. Kafka receives the event
6. Notification Service consumes the event
7. Notification Service logs:
"Sending order confirmation email..."

11.13 Expected Logs

Order Service:

Published OrderCreatedEvent for order: 1001

Notification Service:

```
=====
NOTIFICATION SERVICE
Sending order confirmation email...
Order ID: 1001
Customer ID: 501
Total Amount: 1499.99
=====
```

We have now implemented:

```
Order Service
  |
  | publish
  v
OrderCreatedEvent
  |
  v
Kafka
  |
  | consume
  v
Notification Service
```

12. Compare What We Built in Sessions 6 and 7

Session 6 — Synchronous

```
Order Service
  |
  | OpenFeign
  v
Inventory Service
  |
  | response
  v
Order Service
```

Order Service needs an **immediate** answer.

"Is the product available?"

The caller waits.

Characteristics:

- Caller **knows** the receiver
 - Caller **waits**
 - **Immediate** response
 - Temporal **coupling**
 - Failure **affects** the **current** call
-

Session 7 — Asynchronous

Order Service



Order Service **announces something** that already **happened**.

"An order was created."

The producer **does not** need to **wait** for the notification to be sent.

Characteristics:

- Producer **publishes** an event
 - Producer **does not wait** for the consumer
 - Consumers react **independently**
 - Services are more temporally **decoupled**
 - **Multiple** consumers can subscribe
-

13. ⚖️ Engineering Decision — Sync or Async?

Use this question:

Does the caller need the result immediately to continue the current business operation?

If YES

Consider **synchronous** communication.

Example:

Order → Inventory

"Is stock available?"

Possible implementation:

OpenFeign

If NO

Consider **asynchronous** communication.

Example:

Order → Notification

"An order was created."

Possible implementation:

Kafka Event

Platform Decision Examples

Interaction	Recommended Style	Why
Order → Inventory stock check	Sync	The answer is needed now
Order → Product details	Sync	The caller needs immediate data
Order → Notification	Async	The order should not wait for email
Payment → Analytics	Async	Analytics does not control payment success
Order → Audit	Async	Audit reacts to business events

Do not make the **decision** based on:

- Kafka is more modern.

or:

- REST is easier.

Make the **decision** based on the **required communication semantics**.

The decision is:

What communication semantics does this business interaction require?

14. Important — This Is Not a Saga

What we just implemented is:

Asynchronous Event-Driven Communication

It is **not** automatically a **distributed transaction**.

Kafka ≠ Saga

And:

Async Messaging ≠ Distributed Transaction

Our current flow is:

OrderCreated



Notification Service

is simply **asynchronous event-driven** communication.

There is **no** distributed transaction that needs coordination.

If Notification Service temporarily **fails/unavailable**:

Order is still **valid**.

The notification can be **retried later**.

The notification is an **independent** reaction to an event.

15. Now the Business Problem Changes

Consider a different flow:

Create Order



Reserve Inventory



Process Payment



Confirm Order

Now

These are **no** longer **independent** reactions.

These steps participate in **one** larger business **process**.

Imagine:

Order Created 

Inventory Reserved 

Payment 

The application is still running.

No database is technically corrupted.

But the business state is **inconsistent**.

Inventory has been **reserved** for an order that **cannot** complete.

Now ask:

What happens to the reserved inventory?

We cannot simply leave it reserved.

What should happen now?

16. Why We Cannot Simply Roll Back

In a **monolith** with one database:

BEGIN TRANSACTION

Create Order
Reserve Inventory
Process Payment

Payment fails

ROLLBACK

But in our **microservices** platform:

Order Service

|
└── Order Database

Inventory Service

|
└── Inventory Database

Payment Service

|
└── Payment Database

Each service owns its **own local** transaction.

A transaction inside Order Service **cannot** simply execute:

ROLLBACK

against the **independent databases** owned by Inventory Service and Payment Service.

This is the **distributed transaction** problem.

17. From Kafka Communication to Saga

We already know the mechanics:

Producer
↓
Topic
↓
Consumer

Now we will use those same mechanics for a more **complex** business workflow.

Instead of:

OrderCreated
↓
Notification

we will build:

OrderPlaced
↓
InventoryReserved
↓
PaymentCompleted
↓
OrderConfirmed

And if **payment fails**:

PaymentFailed
↓
ReleaseInventory
↓
OrderCancelled

The technical tools are familiar:

KafkaTemplate
@KafkaListener
Topics
Events

But the business problem is now different.

We are no longer only communicating **asynchronously**.

We are maintaining **consistency** across multiple **independent** services.

This leads us to:

Part 1 — Distributed Transactions: The Problem

and then:

Part 2 — The Saga Pattern

Key Takeaways

Before continuing to Saga, make sure you can explain these three different problems:

OpenFeign

How do I call another service
and wait for the answer?

Kafka

How do I publish an event
without waiting for every interested service?

Saga

How do I maintain business consistency
when multiple independent services participate
in one distributed business process?

These are **not competing technologies**.

They solve different problems.

The progression is:

Session 6

Synchronous Communication



OpenFeign



Session 7

Asynchronous Communication



Kafka Producer + Consumer



Distributed Transaction Problem



Saga Pattern



Saga Choreography



Session 12

Saga Orchestration

Learning Flow

SESSION 6

Inter-Service Communication



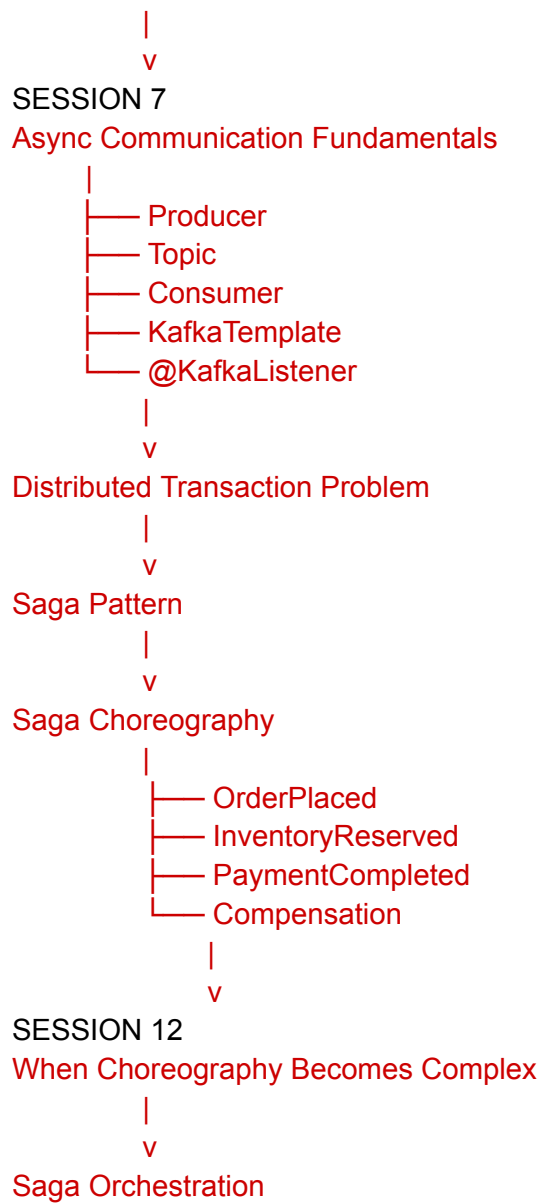
— Sync vs Async Decision



— OpenFeign



— WebClient



Session 7 — Part 0.5: Why Not Use One Distributed Transaction?

From Asynchronous Communication to Distributed Transactions

In the previous section, we used **Apache Kafka** for a simple asynchronous communication flow:



This worked well because the **Order Service** did **not need an immediate response** from the **Notification Service**.

The notification is a side effect of creating the order.

If the notification is delayed for a few seconds, the order itself can still be valid.

But now consider a more difficult business operation.

1. The New Business Problem

Suppose placing an order requires three operations:

1. Create the order.
2. Reserve inventory.
3. Process payment.

In our E-Commerce Platform, these responsibilities belong to different services:

Order Service
|



Each service owns its own database:

Order Service → Order Database

Inventory Service → Inventory Database

Payment Service → Payment Database

Now imagine the following execution:

1. Order created 
2. Inventory reserved 
3. Payment failed 

What is the state of the system?

Order Database
Order = CREATED  Not correct anymore

Inventory Database
Stock = RESERVED  Customer did not pay

Payment Database
Payment = FAILED

The system is now **technically running**, but the business state is inconsistent.

This is called:

Business Inconsistency

The question is:

Can we put all three operations inside one database transaction?

2. Why ACID Transactions Work in a Monolith

In a traditional monolithic application, the Order, Inventory, and Payment operations may use the same database.

For example:

Monolithic Application



Single Database

The application can execute:

```
BEGIN TRANSACTION;
```

```
INSERT INTO orders (...);
```

```
UPDATE inventory  
SET quantity = quantity - 1  
WHERE product_id = 100;
```

```
INSERT INTO payments (...);
```

```
COMMIT;
```

If the payment operation fails:

```
ROLLBACK;
```

The database undoes all previous changes.

The final result is:

- Order created? NO
- Inventory reserved? NO
- Payment completed? NO

Everything succeeds together or everything fails together.

This is the traditional:

ACID Transaction

3. Why the Same Transaction Does Not Naturally Work Across Microservices

In our Microservices Platform, the data is distributed:

Order Service



Inventory Service



Payment Service



There is no single database transaction controlling all three databases.

We cannot simply write:

```
BEGIN TRANSACTION;
```

```
Order Database    → INSERT order
```

```
Inventory Database → UPDATE stock
```

```
Payment Database  → INSERT payment
```

```
COMMIT;
```

because each database has its own independent transaction.

For example:

```
Order DB Transaction
BEGIN
INSERT order
COMMIT
```

Then:

Inventory DB Transaction

BEGIN

UPDATE stock

COMMIT

Then:

Payment DB Transaction

BEGIN

PROCESS payment

ROLLBACK

The first two transactions have already committed.

The Payment Service cannot execute:

ROLLBACK Order Database

ROLLBACK Inventory Database

because it does not own those databases.

This follows one of our core Microservices rules:

Each service owns its own data.

So we need another solution.

One historical solution is:

Two-Phase Commit — 2PC

4. What Is Two-Phase Commit?

Two-Phase Commit, usually abbreviated as **2PC**, is a distributed transaction protocol.

Its purpose is to coordinate one logical transaction across multiple transactional resources.

Instead of allowing each participant to commit independently, a central component coordinates the decision.

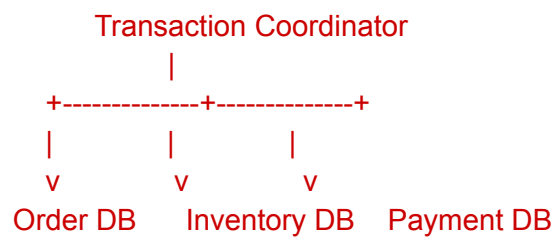
This component is called the:

Transaction Coordinator

The participating systems are called:

Participants

Conceptually:



The coordinator asks all participants to agree before the final transaction is committed.

As the name suggests, the protocol has two phases:

Phase 1 → PREPARE

Phase 2 → COMMIT or ROLLBACK

5. Phase 1 — Prepare

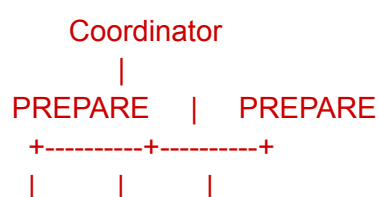
Suppose we want to execute:

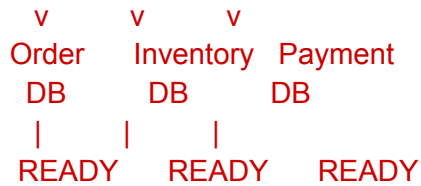
Create Order
Reserve Inventory
Process Payment

The coordinator first asks every participant:

Can you successfully commit this transaction?

Conceptually:





Each participant performs the necessary work but does **not finalize the transaction yet**.

Instead, it responds with something similar to:

- YES — I am prepared to commit

or:

- NO — I cannot commit

For example:

- Order DB → READY
- Inventory DB → READY
- Payment DB → READY

If all participants vote **YES**, the coordinator moves to Phase 2.

6. Phase 2 — Commit

If every participant is prepared:

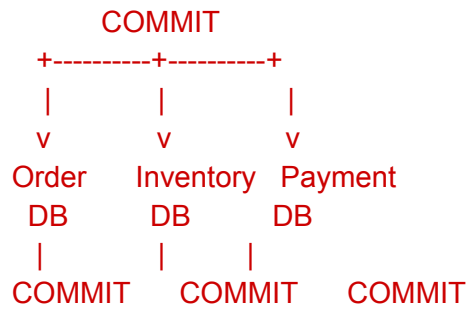
- Order DB → YES
- Inventory DB → YES
- Payment DB → YES

the coordinator sends:

COMMIT

to all participants.

Coordinator
|



The result is:

Order created ✓
 Inventory reserved ✓
 Payment completed ✓

7. What If One Participant Cannot Commit?

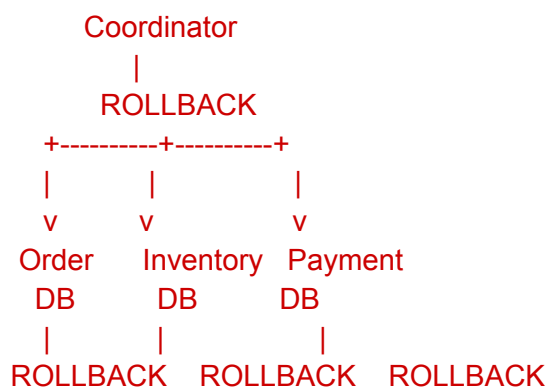
Suppose the Payment participant reports:

- Order DB → READY
- Inventory DB → READY
- Payment DB → FAILED

The coordinator decides:

ROLLBACK

and tells all participants to abort.



Conceptually, this gives us the behavior we want:

All succeed

OR

All fail

So why do we not simply use 2PC everywhere?

8. The Problem with 2PC in Microservices

2PC can provide strong consistency, and it is used in some systems.

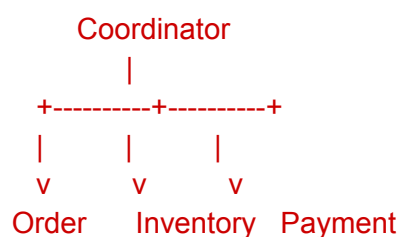
However, it introduces significant coordination and availability costs.

For independently deployable microservices communicating over a network, these costs often conflict with the architecture we are trying to build.

Problem 1 — The Coordinator Becomes Critical

The transaction depends on the coordinator.

Consider:



What happens if the coordinator fails at a critical moment?

For example:

1. Order → PREPARED
2. Inventory → PREPARED
3. Payment → PREPARED
4. Coordinator crashes ❌

The participants may not know the final decision.

Should they:

COMMIT?

or:

ROLLBACK?

Until the transaction outcome is resolved, resources may remain unavailable or locked.

The coordinator therefore becomes a critical dependency in the transaction protocol.

Problem 2 — Participants May Hold Resources While Waiting

During the transaction, participants may need to keep transaction state and resources until the coordinator makes the final decision.

Conceptually:

Order DB

|
└─ Waiting...

Inventory DB

|
└─ Waiting...

Payment DB

|
└─ Waiting...

In a local database transaction, this coordination is fast because everything happens inside one database engine.

Across distributed systems, the participants communicate over a network.

Network communication introduces:

- Latency
- Timeouts
- Partial failures

- Service restarts
- Network partitions

Long coordination can reduce throughput and increase contention.

Problem 3 — Availability Becomes Coupled

Suppose the business transaction requires:

Order Service
Inventory Service
Payment Service

If one required participant is unavailable:

Payment Service = DOWN

the distributed transaction may be unable to complete.

Conceptually:

- Order → READY
- Inventory → READY
- Payment → UNAVAILABLE ✗

Final transaction → CANNOT COMPLETE

The availability of the business operation becomes tightly coupled to all required participants.

This is especially problematic as the system grows.

Problem 4 — Network Latency Becomes Part of the Transaction

A local database transaction may execute entirely inside one database process.

A distributed transaction crosses:

Application



The transaction duration now depends on:

- Network latency
- Slow participants
- Timeout configuration
- Retries
- Coordinator availability

The longer a transaction remains active, the greater the potential cost of holding transactional resources.

Problem 5 — Tight Runtime Coordination

One major goal of Microservices Architecture is independent services.

We want services that can:

- Deploy independently
- Scale independently
- Fail independently
- Recover independently

A distributed transaction introduces strong runtime coordination.

Order

```
|
+---- waits for ----> Inventory
|
+---- waits for ----> Payment
|
+---- coordinated by ----> Transaction Coordinator
```

This can reduce the loose coupling we are trying to achieve.

9. Does This Mean 2PC Is Always Wrong?

No.

That would be an oversimplification.

2PC is a real distributed transaction protocol and can be appropriate in environments where:

- strong consistency is mandatory;
- all participants support compatible transactional protocols;
- the infrastructure is tightly controlled;
- reduced availability or higher coordination cost is acceptable.

However, for many modern Microservices architectures, especially systems designed for:

- Independent deployment
- Horizontal scaling
- Fault isolation
- High availability
- Event-driven communication

2PC is usually not the preferred default.

The better engineering question is not:

Is 2PC technically possible?

The better question is:

Does the consistency guarantee justify the coordination, availability, and scalability cost?

For our E-Commerce Microservices Platform, we will choose a different model.

10. The Microservices Alternative: Local Transactions

Instead of creating one global transaction:

BEGIN GLOBAL TRANSACTION

Order DB
Inventory DB
Payment DB

COMMIT EVERYTHING

each service performs its own:

Local Transaction

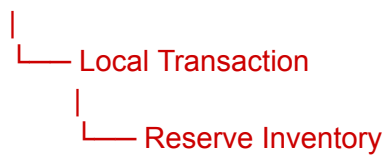
For example:

Order Service



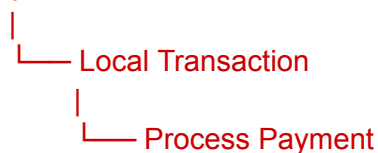
Then:

Inventory Service



Then:

Payment Service



Each service owns and commits its own data.

But this creates a new question:

What happens if a later step fails after earlier services have already committed?

Example:

Create Order  COMMITTED

Reserve Inventory  COMMITTED

Process Payment  FAILED

We cannot perform a global database rollback.

Instead, we perform:

Compensating Actions

11. Compensation Instead of Global Rollback

Suppose this happened:

Order Created
↓
Inventory Reserved
↓
Payment Failed

We cannot execute:

ROLLBACK EVERYTHING

Instead, we perform new business operations that compensate for the previous successful operations.

For example:

PaymentFailed
|
v
ReleaseInventory
|
v

CancelOrder

The complete failure path becomes:

Order Created 

Inventory Reserved 

Payment Failed 



Inventory Released  Compensation



Order Cancelled  Compensation

Notice an important difference.

A database rollback says:

Pretend the transaction never happened.

A compensation says:

The operation happened,
but another business operation corrected its effect.

This distinction is fundamental in distributed systems.

12. From 2PC to Saga

We can now compare the two approaches.

Two-Phase Commit	Saga
One distributed transaction	Sequence of local transactions
Central commit decision	Business steps coordinated through events or commands
Participants wait for transaction outcome	Services commit locally
Strong coordination	Looser service coordination
Global rollback semantics	Compensating actions
Strong consistency focus	Eventual consistency
Infrastructure transaction protocol	Business transaction pattern

The Saga approach accepts an important reality:

In a distributed system, the entire business operation may not become consistent at exactly the same instant.

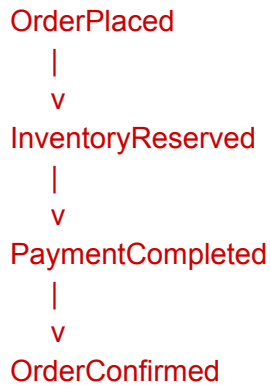
Instead, the system moves through valid intermediate states until it reaches a final consistent state.

This is called:

Eventual Consistency

13. Our E-Commerce Example

Happy Path



Each step is a local transaction.

Order Service
└─ Create Order

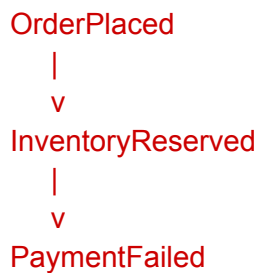
Inventory Service
└─ Reserve Stock

Payment Service
└─ Process Payment

Order Service
└─ Confirm Order

Failure Path

Now suppose payment fails:



The Saga starts compensation:

PaymentFailed
|
v
ReleaseInventory
|
v
OrderCancelled

The final state becomes consistent again:

Order = CANCELLED

Inventory = AVAILABLE

Payment = FAILED

No global database transaction was required.

14. Where Does Kafka Fit?

In the previous section, we already learned the basic Kafka mechanics:

Producer
|
v
Kafka Topic
|
v
Consumer

We used:

Order Service
|
| OrderCreatedEvent
v
Kafka
|
v
Notification Service

That was simple asynchronous communication.

Now we will use the same communication mechanism for a more complex business workflow.

For example:



The important distinction is:

Kafka is the transport mechanism.

Saga is the distributed business transaction pattern.

Kafka does not automatically implement a Saga.

Kafka gives us:

- Event transport
- Durable messages
- Producer/Consumer communication
- Asynchronous delivery

The Saga defines:

- Which business step happens next
 - Which event represents success
 - Which event represents failure
 - Which compensation must execute
-

15. The Learning Progression

We have now built the concepts in the following order:

Session 6

Synchronous Communication

|

v

OpenFeign

|

v

Caller needs the answer NOW

Then:

Session 7 — Part 0

Asynchronous Communication

|

v

Kafka Producer / Consumer

|

v

Order → Notification

Then:

Session 7 — Part 0.5

Distributed Business Problem

|

v

Why one ACID transaction does not naturally span service-owned databases

|

v

2PC

|

v

Why we do not choose it as our default architecture

Now we are ready for:

Session 7 — Saga Pattern

|

v

Local Transactions

+

16. Final Decision for Our Platform

For the E-Commerce Platform, we will use:

- ✗ One global ACID transaction across all services
- ✗ Two-Phase Commit as the default distributed transaction strategy
- ✓ Local transaction inside each service
- ✓ Kafka for asynchronous event transport
- ✓ Saga Pattern for the distributed business workflow
- ✓ Compensation when a later business step fails
- ✓ Eventual consistency across services

The core rule is:

Inside one service and one database: use a local ACID transaction.

Across multiple independently owned service databases: coordinate the business process using a Saga.

17. Before Moving to Saga — Check Your Understanding

Consider this flow:

Order Service
|
v
Inventory Service
|

↓
Payment Service

The first two steps succeed:

Order Created ✓

Inventory Reserved ✓

Then payment fails:

Payment Failed ✗

Answer these questions:

1. Why can the Payment Service not simply roll back the Order Database?
 2. Why can the Inventory Database not remain locked while waiting indefinitely for the Payment Service?
 3. What would 2PC require all participants to do before the final commit?
 4. What happens if the transaction coordinator fails during a distributed transaction?
 5. In a Saga, what should happen to the reserved inventory after **PaymentFailed**?
 6. Is Kafka itself the Saga, or is Kafka only the communication mechanism used by the Saga?
-

18. Key Takeaway

Monolith

↓
↓
One Database
↓
↓
One ACID Transaction
↓
↓
COMMIT or ROLLBACK

Microservices

↓
↓
Multiple Service-Owned Databases
↓
↓

Local Transactions

+

Events

+

Compensation

|

v

SAGA

The Core Message

A distributed business transaction is not one large database transaction.

It is a sequence of local transactions that must be coordinated when success and failure occur across multiple services.

We now have the problem.

We have examined one possible solution: **Two-Phase Commit (2PC)**.

We understand why it is not the preferred default for our Microservices Platform.

Now we are ready to implement the practical pattern used in this course:

The Saga Pattern

OrderPlaced

↓

InventoryReserved

↓

PaymentCompleted

↓

OrderConfirmed

And when something fails:

PaymentFailed

↓

ReleaseInventory

↓

OrderCancelled

Next: Session 7 — Saga Pattern: Distributed Transactions with Kafka and Compensation